Raisonnement sans LLM pour l'audit IoT

Références simples, politiques et vérification de configurations

Projet LANCE

Recherche exploratoire assistée par Codex

Dernière révision : 2026-10-07 14:28 (Europe/Brussels, UTC+02:00)

Statut : revue ciblée et expériences proposées ; aucun gain mesuré

Résumé

Le comité de thèse demande de mesurer l'apport des modèles de langage face à des méthodes plus simples. Cette revue examine le raisonnement défensif sur dossiers et configurations fournis, sans nouvelle collecte réseau. Dix sources primaires permettent de distinguer règles métier et requêtes relationnelles, inférence Datalog, politiques d'autorisation, puis vérification symbolique de configurations réseau. Le D1 à règles déjà documenté dans LANCE constitue le point d'appui ; ce rapport propose d'en adapter les règles à une lecture hors ligne de dossiers, avec normalisation commune, justification et abstention explicite. Une base relationnelle facilite certaines liaisons ; elle ne constitue pas à elle seule un nouveau raisonnement. Les formalismes plus riches sont des options conditionnées par les propriétés étudiées et les données disponibles. Quatre expériences hors ligne isolent moteur, formalisation et information. Le coût humain de préparer les règles et les modèles doit être observé. Aucun article n'a été reproduit et aucun résultat ne démontre encore un avantage ou une équivalence de LANCE.

Table des matières

1 Objectif et méthode de revue **2**

2 Contrat : ce que les données permettent d'affirmer **2**

3 Constats : règles, relations et inférence **3**

4 Politiques d'autorisation et états **4**

5 Vérification statique de configurations réseau **4**

6 Hiérarchie proposée et critères de sélection **5**

7 Quatre expériences hors ligne à préparer **6**

8 Discussion, limites et questions pour les relecteurs **8**

9 Conclusion **8**

Références et portée de lecture **8**

Décision proposée. Valider le D1 existant avant son adaptation hors ligne. Conserver D0 secondaire et justifier chaque ajout par une propriété précise.

1 Objectif et méthode de revue

Question. Parmi les décisions d'interprétation et de vérification documentaire de LANCE, lesquelles peuvent être obtenues par des règles simples, à quelles limites, et quel comparateur permettrait ensuite d'attribuer un effet au LLM ? Le périmètre inclut dossiers figés, exports de configuration, inventaires et politiques fournis par le propriétaire, ainsi que des données synthétiques signalées. Les expériences proposées évaluent des propriétés et des conclusions hors ligne ; elles ne définissent aucun actionneur d'exploitation.

Recherche ciblée. Les requêtes ont porté sur SQL récursif, négation Datalog, provenance Soufflé, OPA/Rego, Cedar, FIREMAN, Header Space Analysis, Batfish et NetPlumber. Les documents ont été consultés le 7 octobre 2026. Les passages exploités dépassent les résumés : sémantique et limites des langages, sections de modélisation, protocoles d'évaluation et discussions des articles. Les références précisent ces périmètres. Les sections non mentionnées ne sont pas réputées avoir été relues intégralement. Les outils documentés ne sont pas installés ni exécutés. Les temps et corpus différents des publications ne sont pas mis en concurrence numérique.

Documents locaux. Lecture des instructions et index du dépôt, du protocole agent et automatisation du 27 septembre et des rapports de méthodes du 7 octobre. Le protocole existant définit D0/D1, distingue attribution et utilité pratique, et propose déjà un contraste sur observations communes. La présente revue approfondit ses références classiques ; elle ne remplace pas ses mesures de campagne. Le guide CLI documente un D1 déjà implémenté pour les phases 3 et 4, analyse et vérification. Le *lecteur à règles* décrit ci-dessous est un adaptateur hors ligne *proposé* de cette référence ; il ne décrit pas une nouvelle capacité déjà livrée. Cette distinction est documentaire : aucun exécutable D1 n'est validé par la présente revue. La base Git est a8dbeb9404f0cad960d61e42f02a1d70ca143a59, avec modifications et conflits locaux. Les empreintes de lecture sont consignées dans sans-llm-raisonnement-sources.json. Le commit seul ne décrit pas cette copie de travail ; aucun comportement exécuté n'est déduit des fichiers en conflit.

2 Contrat : ce que les données permettent d'affirmer

Une déclaration doit identifier la ressource, la propriété, l'identité, le contexte d'autorisation et la période. Pour les propositions suivantes, une observation conserve origine, date, version du producteur, périmètre et transformations. Une identité non documentée ne devient pas « anonyme » et une politique absente ne devient pas « accès interdit ». Une configuration décrivant un droit attendu et une réponse observée décrivent deux objets différents.

Politique proposée de complétude. Chaque dossier déclare les champs indispensables et ce qui est couvert : composants, attributs, période et sous-ensemble de configuration. Une recherche négative ne permet une conclusion que dans un domaine déclaré complet pour cette recherche. Sinon le verdict est *insuffisant*, avec obligation manquante. Il faut conserver des observations positives et négatives explicites. La valeur NULL d'une base, l'absence d'un tuple ou un résultat vide ne garantissent pas cette politique.

Les issues proposées sont *étayé*, *réfuté*, *insuffisant*, avec *conflit* signalé lorsqu'une contradiction persiste. Il reste à préenregistrer la règle de priorité si le format impose un seul verdict. Conserver la contradiction comme donnée est préférable à la perdre par écrasement. Le moteur peut raisonner exactement sur un modèle incorrect ; la correction formelle du calcul ne prouve donc pas la fidélité au terrain.

3 Constats : règles, relations et inférence

3.1 Une référence forte peut rester ordinaire

Lecteur à règles proposé. L'entrée commune est un dossier normalisé ; des règles explicites appliquent obligations de preuve, compatibilité de contexte et complétude. Le résultat cite les artefacts utilisés, les règles déclenchées, les contradictions et les champs manquants. La politique peut tenir dans un module Python ou des tables déclaratives. La lisibilité, la couverture et la maintenance sont des mesures à établir ; le choix du langage ne les garantit pas.

D0 reste utile pour établir le niveau d'une procédure fixe correctement conçue, mais D1 est le contraste principal du protocole existant : état, conditions et abstention sont déjà possibles sans LLM. Dans cette expérience hors ligne, « adaptation » signifie actualiser un verdict ou ses obligations à partir du dossier reçu, sans acquérir de nouvelle observation. Un gain contre D0 seul ne permet pas d'affirmer que les méthodes conditionnelles classiques ont été dépassées.

3.2 SQL : beaucoup de liaisons sont déjà des jointures

Fait sourcé. SQLite fournit jointures, filtrage, agrégation et déduplication. Un filtre WHERE élimine aussi les expressions évaluées à NULL ; une jointure externe peut donc perdre ses lignes incomplètes si le filtre est mal placé. Les expressions de table communes récursives permettent également de parcourir des graphes ; l'exemple officiel utilise UNION pour éviter une récursion cyclique répétitive [1].

Proposition. Relier une observation à la bonne déclaration via dossier, ressource, identité et période ; dédupliquer ; lister les obligations non satisfaites ; repérer deux observations incompatibles. Ces fonctions peuvent être des jointures et des règles, sans base de graphes ni LLM. Les noms des tables et identifiants ne doivent pas contenir les labels cachés. Les citations doivent remonter aux observations et pas seulement à une ligne agrégée. SQL ne résout pas automatiquement l'ambiguïté d'une identité ou l'intention métier.

Une variante relationnelle qui ajoute des champs absents du lecteur à règles améliore l'information disponible. Celle qui ajoute une obligation améliore la politique. Pour isoler la représentation, ce lecteur doit pouvoir consulter ces mêmes champs et appliquer ces mêmes obligations. Si les verdicts restent identiques, une réduction du temps de revue ou des erreurs d'attribution peut néanmoins être utile.

3.3 Datalog/Soufflé : justifier la récursion et sa provenance

Faits sourcés. Soufflé exprime relations typées et règles, dont une récursion vers un point fixe et une négation stratifiée [2]. L'article de Zhao et collaborateurs explique des arbres de dérivation construits à partir d'annotations de règle et de hauteur. Il examine aussi les explications de non-dérivation ; la négation signifie absence de tuple dans l'instance. L'évaluation porte sur analyses de programmes Java, avec un coût supplémentaire de provenance [3].

Proposition. Envisager Datalog pour plusieurs règles interdépendantes et une dérivation récursive difficile à maintenir dans le lecteur à règles. Un graphe de dépendances de preuves convient à cette question. La récursion seule ne suffit pas à choisir Datalog puisque SQL en fournit également. Une dérivation explique les règles appliquées ; elle ne certifie ni les données initiales ni l'intention du propriétaire.

4 Politiques d'autorisation et états

4.1 Formaliser l'attendu avant d'interpréter l'observé

Proposition. Pour les cas où les droits sont centraux, commencer par une table lisible : principal, rôle ou attribut, ressource, opération autorisée, conditions et version de politique. Une fonction déterministe suffit souvent sur un domaine fini. Ajouter un état uniquement si la propriété le nécessite, par exemple un rôle révoqué ou une autorisation expirée. Une transition est alors un changement documentaire d'état, pas une instruction à exécuter sur un équipement.

Le test reçoit deux couches séparées : politique attendue fournie et trace de comportement fournie. Un verdict de conformité compare les deux pour la même identité, la même ressource et le même instant. Il doit rester insuffisant si l'attribut nécessaire, la portée ou la version sont inconnus. Déduire une politique attendue uniquement depuis le comportement observé ferait disparaître les écarts que l'on cherche à mesurer.

4.2 OPA/Rego : politique déclarative générale

Fait sourcé. Rego calcule des décisions sur des données structurées. Sa documentation distingue valeurs non définies, défauts explicites, négation et quantification universelle. Elle explique qu'une condition satisfaite pour un élément ne prouve pas qu'elle l'est pour tous [4].

Proposition. OPA convient comme variante de conformité documentaire si l'équipe souhaite exprimer et versionner des règles indépendamment du code. Le moteur recevrait uniquement des données locales fournies. La forme proposée du résultat conserve verdict, motifs, références et obligations manquantes ; elle exige une règle d'abstention au-dessus des valeurs booléennes. Une comparaison avec le lecteur à règles doit garder les prédicats métier identiques : traduire une bonne politique dans Rego puis la comparer à une mauvaise politique Python ne mesurerait pas la valeur du langage. Le risque principal est le coût de traduction, de revue et de mise à jour, encore inconnu pour LANCE.

4.3 Cedar : un modèle centré sur les droits

Fait sourcé. Cedar structure l'autorisation autour du principal, de l'opération, de la ressource et du contexte. Les interdictions applicables l'emportent sur les permissions ; l'absence de permission applicable entraîne un refus. L'article étudie validation typée et compilation symbolique pour comparer des politiques. Son évaluation temporelle utilise trois applications et leurs données en mémoire ; elle exclut certaines étapes d'initialisation et de récupération [5].

Proposition. Réserver Cedar à un sous-corpus où une spécification d'autorisation est disponible et où relations entre entités et exceptions justifient ce langage. Il peut fournir un évaluateur attendu ou un comparateur de deux versions de politique. Un refus par défaut du moteur d'autorisation est une décision d'application ; il ne devient pas une preuve que le dossier d'audit démontre ce refus dans le déploiement réel. La version du moteur et le schéma de chaque expérience devront être figés.

Choix proposé. Ne pas démarrer simultanément OPA et Cedar. Tester d'abord une table de droits et le lecteur à règles proposé. Choisir ensuite un langage si une propriété concrète manque ou si son coût de maintenance semble acceptable, puis mesurer ce coût. Une spécification produite à la main constitue un apport humain qui doit être compté pour tous les comparateurs qui la reçoivent.

5 Vérification statique de configurations réseau

Les approches suivantes forment une piste distincte : elles exploitent des configurations et des propriétés explicites. Elles ne constituent pas des remplaçants directs de l'interprétation de dossiers

incomplets. Leur intérêt pour LANCE dépend d'abord de la présence d'exports fidèles et d'un domaine de syntaxe couvert. Une comparaison sur des captures réseau seules serait inadaptée.

5.1 FIREMAN : anomalies de règles et intention explicite

Fait sourcé. FIREMAN modélise listes et chaînes de règles, puis ensembles de paquets avec diagrammes de décision binaires. Il recherche violations de politique, incohérences et inefficacités. Sa discussion distingue anomalie formelle et intention de l'administrateur : certains résultats restent des avertissements. L'évaluation examine des configurations disponibles de plusieurs types de pare-feu [6].

Proposition. Utiliser cette famille pour définir un petit ensemble de propriétés de configuration hors ligne : règle masquée, traitement contradictoire, permission dépassant une politique fournie. Commencer par les règles et leur sémantique déterministe, pas par la réimplémentation de l'ensemble du toolkit. Une redondance volontaire pour défense en profondeur ne doit pas recevoir automatiquement le label « vulnérabilité ». Le modèle doit préserver l'ordre, les actions et les règles implicites effectivement retenues.

5.2 Header Space Analysis : ensembles et transformations

Fait sourcé. HSA représente les en-têtes comme espaces de bits et compose des fonctions de transfert, afin d'examiner accessibilité, boucles et isolation. L'article évalue le réseau Stanford et d'autres configurations. Sa discussion limite l'analyse aux propriétés du modèle statique : elle n'explique pas la construction des tables et traite mal les changements plus rapides que la collecte de snapshots [7].

Proposition. Une propriété d'isolation sur export complet pourrait motiver un modèle ensembliste. Sur une simple relation d'adjacence, une fermeture transitive SQL suffit souvent. Les transformations d'en-tête et la priorité de règles rendent un modèle de paquets plus pertinent ; l'adjacence seule ne prouve alors pas l'accessibilité. La propriété reste « dans le modèle fourni », avec hypothèses et fonctionnalités omises visibles.

5.3 Batfish et NetPlumber : deux besoins différents

Faits sourcés. L'article Batfish 2015 décrit un modèle déclaratif du plan de contrôle, un plan de données dérivé et des propriétés vérifiées avec solveur. L'évaluation concerne deux réseaux universitaires ; les limites incluent environnements incomplets, fonctionnalités non modélisées et défauts matériels ou logiciels hors modèle [8]. La documentation pybatfish 0.36.0 décrit des statuts et avertissements de parsing [9]. NetPlumber met à jour un graphe de dépendances entre règles après modification ; ses expériences portent sur trois réseaux. Ses limites incluent états dynamiques de middleboxes et coût des modifications de liens [10].

Proposition. Batfish est une option d'analyse d'exports compatibles, après contrôle des avertissements. NetPlumber devient pertinent seulement si une question scientifique porte sur recalcul incrémental ; un audit ponctuel n'en justifie pas le coût d'intégration. L'article historique et la version courante d'un logiciel ne sont pas assimilés : le futur manifeste devra figer versions du serveur, client et fonctionnalités réellement utilisées.

6 Hiérarchie proposée et critères de sélection

Le tableau 1 distingue couverture et travail d'intégration. Les priorités sont des appréciations argumentées, sans estimation chiffrée d'heures ni résultat de benchmark.

Méthode	Pertinence / couverture	Intégration à prévoir	Priorité proposée
D0, procédure fixe	Contrôles documentaires routiniers, contraste de départ.	Checklist utile, sorties et échecs conservés.	Secondaire ; ne pas en faire le seul adversaire du LLM.
Lecteur à règles, adaptation proposée de D1	Obligations connues, contexte, justification, abstention.	Schéma commun et règles revues sur développement.	Référence principale hors ligne, à préparer.
SQL et relations	Attribution, complétude, déduplication ; récursion simple.	Clés, contraintes, index, provenance des agrégations.	Ajouter si dossier volumineux ou relations mal tracées.
Datalog / Soufflé	Inférences récursives interdépendantes, dérivations.	Traduction typée, stratégie de négation, restitution.	Sous-corpus justifiant la récursion ; pas de gain présumé.
Table de droits, puis OPA ou Cedar	Conformité à une politique fournie.	Formalisation métier, identités et versions ; tests de traduction.	Table d'abord ; un langage ensuite si nécessaire.
FIREMAN / HSA, Batfish	Propriétés de configuration et d'isolation dans un modèle.	Exports complets, parseurs, sémantique, exclusions.	Piste séparée sous condition de données et compatibilité.
NetPlumber	Vérification incrémentale après changements.	Événements et dépendances fiables, gestion du modèle.	Hors premier pilote ponctuel.
Contraintes symboliques	Équivalence ou propriété sur un domaine explicite.	Encodage, limites du solveur, contre-exemples.	Après échec motivé d'une énumération finie ou d'une règle simple.

Table 1 – Choix de recherche proposés. La couverture désigne les propriétés du modèle ou du dossier, pas une couverture exhaustive d'un réseau réel.

Contraintes et symbolique. Pour un domaine local fini d'identités, ressources et états, comparer les sorties par énumération peut être plus facile à relire qu'un encodage SMT. Un solveur devient une option lorsque l'espace de configurations ou les propriétés dépassent cette énumération. Ses issues *unknown*, timeout ou erreur doivent rester distinctes d'une preuve. Un contre-exemple doit être interprété et sa faisabilité vérifiée dans le modèle. Il ne constitue pas une observation terrain. Ce choix est une inférence de conception à partir des exigences de formalisation, pas une performance mesurée.

Apprentissage classique sans LLM. Une classification supervisée ou un score d'anomalie pourrait être ajouté ultérieurement si l'on dispose de labels indépendants, représentatifs et stables. Pour les propriétés précises retenues ici, règles et modèles sont le premier choix proposé. L'apprentissage ajoute séparation des données, réglage de seuils et risque de dépendance aux artefacts du corpus ; sans jeu justifié, le placer dans le premier pilote disperserait la recherche. Ce rapport n'effectue pas une revue exhaustive de l'apprentissage classique et n'en conclut pas l'inutilité.

7 Quatre expériences hors ligne à préparer

Principe commun. Les entrées, obligations et références cachées sont figées avant les résultats. Le jeu de développement sert à corriger parseurs, règles et prompts ; le test réservé demeure séparé par famille d'origine. Une variante d'un même générateur ne constitue pas une famille indépendante. Les comparateurs n'accèdent ni aux labels ni aux dossiers de correction.

Porte d'acceptation avant le test réservé. Sur développement uniquement, faire relire la couverture du lecteur à règles : chaque critère retenu dispose d'une règle explicite, d'un contexte applicable et d'un exemple positif, d'un négatif proche et d'un cas incomplet ou inconnu. Vérifier les

justifications, références aux données, contradictions et obligations manquantes, ainsi que l'absence d'utilisation des identifiants ou labels cachés. Fixer avant cette revue les critères d'acceptation et les écarts admissibles sur développement ; consigner toute lacune et sa correction avec le temps humain actif. Un relecteur distinct des auteurs accepte la couverture ou demande de reprendre le développement. Puis figer entrées publiques, règles, schéma, normalisation, obligations, prompts éventuels, budgets et métriques avant d'ouvrir le test. Les lacunes découvertes au test restent des résultats ; elles ne sont pas corrigées pour recalculer silencieusement le même contraste. Une nouvelle version exige un nouveau gel et un nouveau test indépendant, en conservant les résultats de la version précédente. Cette porte qualifie une référence forte pour le périmètre annoncé ; elle ne garantit pas sa performance future.

T1 – Interprétation de dossiers et attribution. Comparer le lecteur à règles proposé puis ce même lecteur avec index relationnel ; ajouter un lecteur LLM recevant les mêmes observations structurées, si l'on souhaite isoler H4 du protocole existant. Les cas couvrent soutien, réfutation, données absentes, mauvaise identité, ressource différente, observations anciennes, contradictions et doublons. L'oracle distingue propriété réelle connue et soutien accessible au dossier. Hypothèse : l'index peut améliorer l'attribution ou la relecture ; une égalité des verdicts est attendue si règles et information sont déjà équivalentes.

T2 – Conformité à une politique d'autorisation. Sur politiques et traces synthétiques fournies, comparer fonction déterministe sur table de droits et un évaluateur OPA *ou* Cedar. Donner au lecteur LLM le même schéma et la même politique explicite. Inclure permission, refus explicite, révocation documentée, rôle absent et version indéterminée. L'oracle de politique est établi par revue indépendante et petit domaine énumérable. Séparer exactitude de la traduction et exactitude de la décision. Une comparaison entre politique naturelle pour le LLM et politique formelle pour le lecteur à règles est une étude d'utilité de bout en bout, avec coûts humains ; elle n'isole pas le seul moteur.

T3 – Propriétés de configuration réseau. Sur exports synthétiques ou autorisés et figés, étudier une propriété d'isolation ou de conformité de règles. La référence simple modélise précisément le sous-ensemble retenu. Une variante statique plus riche reçoit la même information, ou ses entrées supplémentaires sont déclarées comme facteur distinct. Contrôler cas valide, violation locale, fonctionnalités non couvertes, export incomplet et version ancienne. Aucun verdict de modèle n'est compté comme preuve d'accès réel. Batfish n'est retenu qu'après vérification de compatibilité des exports et politique préenregistrée de traitement des avertissements.

T4 – Récursion et maintenance des règles. Préparer des dossiers où les dépendances documentaires forment plusieurs niveaux. Comparer SQL récursif et Datalog à prédicats métier égaux. Tester cycles, doublons, entrée manquante, contradiction et retrait d'une observation entre deux versions. La question est la correction et la facilité de restitution/maintenance. Ajouter Datalog sans cas qui distingue sa valeur ne fournirait qu'une traduction supplémentaire.

Mesures proposées. Exactitude des verdicts et motifs ; précision et rappel d'étayage ; confirmations abusives ; attribution valide ; couverture décidée ; résultats insuffisants et conflits ; complétude d'exécution ; temps écoulé, mémoire, calcul et coût du modèle. Publier les comptes et tous les essais prévus, y compris erreurs et délais dépassés. Une amélioration de précision obtenue en s'abstenant davantage doit rester visible.

Consigner séparément heures actives de formalisation, annotation, traduction, développement, correction et maintenance ; ne pas les inventer depuis des tokens ou des temps machine. Apparier chaque méthode sur le même dossier, puis analyser par famille. Le pilote est descriptif ; taille du test, effet minimal utile et marges des garde-fous se fixent avant le test réservé. Une incertitude compatible avec gain et perte conduit à un résultat indéterminé.

8 Discussion, limites et questions pour les relecteurs

La revue est ciblée et ne couvre pas toutes les approches de vérification formelle ni tous les outils d'audit. Les expériences originales ne sont pas reproduites. Les articles anciens apportent des mécanismes et des limites, pas une preuve de compatibilité des outils actuels avec LANCE. Les performances rapportées dans des corpus hétérogènes ne permettent aucun classement global. Une documentation consultée sans version figée devra être remplacée par une référence de logiciel reproductible avant expérimentation.

Les questions prioritaires pour une contre-revue sont : le lecteur à règles couvre-t-il les obligations métier sans recevoir les solutions cachées ? L'effet annoncé vient-il du moteur, d'une meilleure spécification ou de données supplémentaires ? Un parseur commun masque-t-il des erreurs communes ? La distinction inconnu/faux survit-elle jusqu'aux mesures ? La politique attendue a-t-elle été inventée depuis l'observation ? Les familles de test sont-elles suffisamment distinctes ? Le coût humain d'une formalisation est-il enregistré dans tous les groupes ?

L'oracle ne doit pas être seulement le moteur comparé ou un validateur commun. Des annotations indépendantes, désaccords conservés et cas finis relus offrent une référence plus contestable et vérifiable. Un défaut partagé entre modèle, normalisation et oracle peut néanmoins subsister. Des cas synthétiques sont adaptés à une erreur ciblée, mais ne donnent pas sa fréquence terrain.

9 Conclusion

La priorité scientifique est de valider la couverture du D1 existant et de préparer son adaptateur hors ligne à règles, avec abstention, contexte et justification. La porte d'acceptation puis le gel précèdent le test réservé. Les jointures et règles couvrent déjà beaucoup de problèmes d'attribution. Datalog, OPA/Cedar et les vérificateurs statiques répondent à des propriétés différentes ; ils s'ajoutent lorsque les données et les cas le justifient.

Le premier travail testable est T1 sur dossiers communs, puis T2 si des politiques d'autorisation sont disponibles. T3 exige des exports compatibles et T4 une difficulté récursive réelle. Une absence de gain du LLM, un gain limité à la restitution ou une référence classique meilleure restent des conclusions recevables. À ce stade, seuls la revue et ces protocoles sont produits.

Références

[1] SQLite. *SELECT ; The WITH Clause*, documentation officielle non figée, consultée le 7 octobre 2026. SELECT et WITH. Lecture : jointures, filtres, NULL, déduplication ; exemples récursifs, cycles. Pas de comparaison de performance SQL reproduite.

[2] Projet Soufflé. *Datalog Language*, documentation officielle non figée, consultée le 7 octobre 2026. Référence du langage. Lecture : relations, types, clauses et négation ; aucune version exécutable testée. Les détails de provenance proviennent de la référence suivante.

[3] D. Zhao, P. Subotic et B. Scholz. *Provenance for Large-scale Datalog*, arXiv :1907.05045v1, juillet 2019. Version primaire consultée. Lecture ciblée : sections 3.1–3.4, passages 4.2 et 5 ; sémantique, arbres de preuve, non-dérivation, expérimentation Doop/DaCapo. Preuves mathématiques non vérifiées intégralement ; aucune reproduction.

[4] Open Policy Agent. *Policy Language*, documentation officielle non figée, consultée le 7 octobre 2026. Rego. Lecture ciblée : valeurs non définies, négation, quantification, défauts. Les exemples de documentation ne constituent pas un benchmark LANCE.

[5] J. W. Cutler et collaborateurs. *Cedar : A New Language for Expressive, Fast, Safe, and Analyzable Authorization*, version étendue arXiv :2403.04651v1, 7 mars 2024 ; publication OOPSLA1, avril 2024. Version consultée. Lecture ciblée : sections 2, 3.3/3.5, 4.3 et 5 ; sémantique, limites du modèle, protocole d'évaluation. Preuves Lean et artefact non vérifiés.

[6] L. Yuan et collaborateurs. *FIREMAN : A Toolkit for FIREwall Modeling and ANalysis*, IEEE Security and Privacy, 2006, p. 199–213. PDF des auteurs. Lecture ciblée : sections 2, 4.1, 4.2.2–4.3, 5.1–5.3 ; pages PDF 2–3, 7–11. Modélisation, intention et cadre d'évaluation, sans reproduction.

[7] P. Kazemian, G. Varghese et N. McKeown. *Header Space Analysis : Static Checking for Networks*, NSDI 2012, p. 113–126. PDF universitaire. Lecture ciblée : sections 2, accessibilité section 4, début section 7 et section 8 ; pages PDF 2–3, 5–6, 9–10, 13. Instantanés et hypothèses, sans reproduction.

[8] A. Fogel et collaborateurs. *A General Approach to Network Configuration Analysis*, NSDI 2015, p. 469–483. PDF d'auteur. Lecture ciblée : sections 3, 6 et 7 ; pages PDF 4–6, 9–13 ; mécanisme, environnement, restrictions et protocoles. Article historique ; il ne décrit pas toutes les fonctionnalités d'un logiciel actuel.

[9] Projet Batfish. *Snapshot Input*, documentation pybatfish affichée 0.36.0, consultée le 7 octobre 2026. Entrées et parsing. Lecture : statuts de fichier et avertissements ; serveur non testé, URL latest évolutive. Une version de campagne devra être figée.

[10] P. Kazemian et collaborateurs. *Real Time Network Policy Checking Using Header Space Analysis*, NSDI 2013, p. 99–111. PDF USENIX. Lecture ciblée : section 3.1, protocole section 6 et discussion section 7 ; pages PDF 2–3, 9–12. Incidence des mises à jour et limites ; aucune reproduction.